

1 json

- **Purpose:** Read and write JSON files.
- **Use in your code:** Load the chatbot dataset containing intents, patterns, and tags.

2 numpy (np)

- **Purpose:** Handle numerical operations and arrays efficiently.
- **Use in your code:** Convert labels and predictions into arrays; useful for model input and evaluation.

3 pickle

- **Purpose:** Save and load Python objects.
- **Use in your code:** Save the tokenizer and classes so you can reuse them later without retraining.

4 tensorflow.keras

- **Purpose:** Build and train deep learning models.
- **Modules used:**
 - **Sequential** → Stack layers to build a model.
 - **Dense** → Fully connected layer (used for output).
 - **LSTM** → Learn patterns in sequences (text).
 - **Embedding** → Convert word indices to dense vectors (word meaning).
 - **Dropout** → Prevent overfitting by randomly turning off neurons.
 - **Tokenizer** → Convert text into sequences of numbers.
 - **pad_sequences** → Make sequences the same length.
 - **to_categorical** → Convert labels to one-hot encoding.
 - **Adam** → Optimizer to adjust model weights.

5 sklearn.metrics

- **Purpose:** Evaluate machine learning models.
- **Functions used:**
 - **accuracy_score** → Compute overall accuracy.
 - **classification_report** → Show detailed metrics (precision, recall, F1-score) for each class.

In **one line**:

json → load data, numpy → arrays, pickle → save/load objects, keras → build/train LSTM model, sklearn → evaluate performance.

1 json

```
import json
```

- **json.load()** → Reads JSON data from a file into a Python dictionary.
- **json.dump()** → (Not used in your code) Write Python objects into a JSON file.

2 numpy

```
import numpy as np
```

- **np.array()** → Converts a list to a NumPy array for easier computation.
- **np.argmax()** → Returns the index of the maximum value in an array (used to get predicted class).

3 pickle

```
import pickle
```

- **pickle.dump(obj, file)** → Save a Python object (tokenizer or class list) to a file.
- **pickle.load(file)** → Load a saved Python object from a file (not in your code, but used during prediction).

4 TensorFlow / Keras

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, LSTM, Embedding, Dropout
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.optimizers import Adam
```

Imported Classes/Functions and Use

Imported	Use in Your Code
Sequential	Create a linear stack of neural network layers.
Dense	Fully connected layer, used for output predictions.
LSTM	Long Short-Term Memory layer, learns patterns in text sequences.
Embedding	Convert word indices into dense vector representations.
Dropout	Prevent overfitting by randomly ignoring neurons during training.
Tokenizer	Convert text into sequences of word indices.
pad_sequences	Make all sequences same length for LSTM input.
to_categorical	Convert numeric labels into one-hot vectors for multi-class classification.
Adam	Optimizer for training, adjusts weights efficiently.

5 scikit-learn

```
from sklearn.metrics import classification_report, accuracy_score
```

- **accuracy_score(y_true, y_pred)** → Calculates overall prediction accuracy.
- **classification_report(y_true, y_pred, target_names=classes)** → Gives precision, recall, F1-score per class.

Summary in One Line

- json → load JSON data
- numpy → arrays & array operations
- pickle → save/load Python objects
- Sequential + Dense + LSTM + Embedding + Dropout → build LSTM model
- Tokenizer + pad_sequences → preprocess text
- to_categorical → one-hot encode labels
- Adam → optimizer
- accuracy_score + classification_report → evaluate model